

# Labo 02 — Labo pratique : disséquer une image, monter un registry

*Objectif : manipuler tags, couches et digests, puis publier une image dans un registry privé que vous ferez tourner vous-même — et découvrir au passage pourquoi Podman vous fait écrire les noms en entier.*

**Prérequis** — Labo 01 terminé (Podman rootless sous WSL, `systemd` actif). Le port `5000` doit être libre (`ss -lntp | grep :5000` ne doit rien renvoyer).

**Fichiers fournis** — `files/Dockerfile` (deux lignes, expliquées au labo 04 ; ici on s'en sert seulement comme générateur d'images).

## Étape 1 — Lire un nom d'image

```
podman pull nginx:alpine
podman pull alpine
```

**Observez** `Resolved "nginx" as an alias`, puis `Trying to pull docker.io/library/nginx:alpine...`, les lignes `Copying blob`, et pour `alpine` un simple identifiant : l'image est déjà là depuis le labo 01.

```
podman image inspect --format '{{.RepoTags}}' nginx:alpine
podman image inspect --format '{{.Digest}}' nginx:alpine
```

**Observez** d'un côté `[docker.io/library/nginx:alpine]` — le nom **complet**, que vous n'avez pas tapé — de l'autre `sha256:1f25fedd50aec27413031afb...`.

*Explication.* `nginx:alpine` est un nom lisible et déplaçable ; le digest est l'identité réelle et permanente du contenu. Podman affiche toujours le nom complet : il n'y a pas de « nom court » dans son stockage, seulement dans votre commande.

Essayez maintenant un nom qui n'est pas dans la liste d'alias :

```
grep -c '=' /etc/containers/registries.conf.d/000-shortnames.conf
grep -E '^s*"(\alpine|nginx|eclipse-temurin)"' /etc/containers/registries.conf.d/000-shortnames.conf
grep unqualified-search-registries /etc/containers/registries.conf
```

**Observez** que `alpine` et `nginx` ont un alias, `eclipse-temurin` non, et que la liste de recherche d'Ubuntu ne contient que `docker.io` — c'est pourquoi `podman pull eclipse-temurin:21-jre-alpine` fonctionne malgré tout chez vous, alors qu'il poserait une question sur Fedora.

### PODMAN

Un nom court est une commodité de terminal, pas une pratique d'entreprise. Dans un Dockerfile ou un script, écrivez `docker.io/library/eclipse-temurin:21-jre-alpine` : le résultat ne dépendra plus de la configuration de la machine qui exécute.

## Étape 2 — Lister ses images

```
podman images
```

**Observez** les colonnes `REPOSITORY / TAG / IMAGE ID / CREATED / SIZE`, et des dépôts écrits en entier : `docker.io/library/nginx`, `docker.io/library/alpine`.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman images --format '{{.Repository}}:{{.Tag}}'
```

*Explication.* Prenez l'habitude de `--format` : il rend vos commandes indépendantes des changements de présentation, et exploitables en script.

## Étape 3 — Les couches, et où passe le poids

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'
```

**Observez** une seule grosse couche ( `50.7MB`, l'installation d'nginx), quelques petites couches `COPY` de scripts, une couche `8.7MB` (Alpine) tout en bas, et de nombreuses lignes à `0B`.

*Explication.* Les lignes à `0B` sont des instructions de **métadonnées** : `ENV`, `CMD`, `EXPOSE`, `ENTRYPOINT`, `STOPSIGNAL`. Elles ne créent aucun fichier. Cette commande est votre premier réflexe quand une image est anormalement lourde : la ligne coupable saute aux yeux.

Podman sait aussi présenter les couches comme un arbre, avec leur image d'origine :

```
podman image tree nginx:alpine
```

**Observez** la première couche marquée `Top Layer of: [docker.io/library/alpine:latest]` : `nginx:alpine` est **construite sur** l'image `alpine` que vous avez déjà — cette couche de 8,7 Mo n'est stockée qu'une fois.

```
podman system df
podman system df -v | head -n 12
```

**Observez** dans le mode verbeux la colonne `SHARED SIZE` : `8.698MB` pour `nginx` et pour `alpine`, la même couche comptée dans les deux.

## Étape 4 — `podman tag` ne copie rien

```
podman tag nginx:alpine mon-nginx:v1
podman tag nginx:alpine mon-nginx:preprod
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}' | grep nginx
```

**Observez** trois lignes... avec **le même IMAGE ID** — et vos deux nouveaux noms préfixés de `localhost/`.

```
podman rmi mon-nginx:v1
```

**Observez** la sortie : uniquement `Untagged: localhost/mon-nginx:v1`. Aucun `Deleted:`.

```
podman rmi mon-nginx:preprod
```

**Observez** encore `Untagged:` seulement — car `docker.io/library/nginx:alpine` désigne toujours l'image.

*Explication.* Un tag est une référence. Tant qu'il reste un nom, les données restent. C'est pourquoi « j'ai fait des `rmi` et je n'ai pas récupéré de place » est une plainte fréquente et parfaitement normale. Quant au `localhost/` : une image que vous nommez sans registry n'appartient à aucun registry, et Podman le dit.

## Étape 5 — Construire deux versions d'une même image

Copiez le fichier fourni dans un dossier de travail :

```
mkdir -p ~/labo-docker/02 && cd ~/labo-docker/02
cp <chemin-du-labo>/files/Dockerfile .
cat Dockerfile
```

```
podman build -t demo-couches:1.0 .
podman history demo-couches:1.0 --format 'table {{.ID}}\t{{.Size}}\t{{.CreatedBy}}'
```

**Observez** les lignes `STEP 1/2`, `STEP 2/2`, `COMMIT demo-couches:1.0`, `Successfully tagged localhost/demo-couches:1.0`, puis trois couches : votre `RUN` ( `2.05kB` ), le `CMD` de l'image de base ( `0B` ), et le système de fichiers Alpine ( `8.7MB` ), marqué `<missing>` car cette couche appartient à une autre image.

Modifiez la version puis reconstruisez sur **le même tag** :

```
sed -i 's/version 1/version 2/' Dockerfile
podman build -t demo-couches:1.0 .
podman run --rm demo-couches:1.0 cat /version.txt
```

**Observez** `version 2`, et un nouvel `IMAGE ID` pour le même tag.

```
podman images --filter dangling=true
```

**Observez** une ligne `<none> <none>` avec l'**ancien** `IMAGE ID` : c'est l'image *dangling*, celle qui a perdu son nom. Elle occupe toujours 8,7 Mo (partagés, en l'occurrence).

*Explication.* Le tag `demo-couches:1.0` a été **déplacé** vers une nouvelle image : exactement le scénario de la question 2. Rien n'avertit l'utilisateur. Supprimez le résidu par son identifiant :

```
podman rmi $(podman images --filter dangling=true -q)
```

## Étape 6 — Monter un registry privé

Un registry n'est rien d'autre qu'un conteneur :

```
podman run -d -p 5000:5000 --name registry-labo registry:2
podman ps --filter name=registry-labo
curl -s http://localhost:5000/v2/_catalog
```

**Observez** `0.0.0.0:5000->5000/tcp` dans la colonne `PORTS`, puis `{"repositories":[]}` : le registry est vide et fonctionnel.

#### → WINDOWS / WSL

Ce port 5000 est publié dans la VM WSL, mais Windows le voit aussi : ouvrez `http://localhost:5000/v2/_catalog` dans votre navigateur Windows. WSL 2 relaie automatiquement les ports écoutés dans Linux vers `localhost` côté Windows (*localhost forwarding*). C'est ce qui vous permettra de tester le front Angular depuis Edge ou Chrome dans les labos suivants.

Publiez-y votre image :

```
podman tag demo-couches:1.0 localhost:5000/socle/demo:1.0
podman push localhost:5000/socle/demo:1.0
```

**Observez** l'échec :

```
Error: ... pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

*Explication.* Votre registry parle HTTP ; Podman exige HTTPS par défaut, **même pour localhost** — là où Docker fait une exception silencieuse. Pour un registry de test, on le dit explicitement :

```
podman push --tls-verify=false localhost:5000/socle/demo:1.0
curl -s http://localhost:5000/v2/_catalog
curl -s http://localhost:5000/v2/socle/demo/tags/list
```

**Observez** les `Copying blob`, `Writing manifest to image destination`, puis `{"repositories":["socle/demo"]}` et `{"name":"socle/demo","tags":["1.0"]}`.

#### → SÉCURITÉ

L'alternative est de déclarer le registry dans `~/.config/containers/registries.conf` (`[[registry]]`, `location = "localhost:5000"`, `insecure = true`). C'est pratique sur un poste de développement, et dangereux partout ailleurs : un registry « insecure » est un registry dont on ne vérifie ni l'identité ni le chiffrement, donc dans lequel un attaquant sur le réseau peut substituer une image. En entreprise, un registry a un certificat, point.

Vous venez de reproduire, en trois commandes, ce que fait la CI de votre entreprise. Vérifiez le transfert différentiel :

```
podman tag demo-couches:1.0 localhost:5000/socle/demo:1.1
podman push --tls-verify=false localhost:5000/socle/demo:1.1
```

**Observez** que les mêmes blobs sont cités mais que le transfert est instantané : le registry les possède déjà, seul le manifeste est écrit.

## Étape 7 — Tirer par digest

Récupérez le digest tel que le registry le connaît :

```
curl -sI -H "Accept: application/vnd.oci.image.manifest.v1+json" \
  http://localhost:5000/v2/socle/demo/manifests/1.0 | grep -i docker-content-digest
```

**Observez** une ligne `Docker-Content-Digest: sha256:239accdd...`. Copiez cette valeur.

```
podman rmi localhost:5000/socle/demo:1.0
podman pull --tls-verify=false localhost:5000/socle/demo@sha256:<collez_ici>
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Digest}}' | grep demo
```

**Observez** que l'image est tirée et que `podman image inspect --format '{{.Digest}}' demo-couches:1.0` donne exactement la valeur que vous venez de coller.

*Explication.* C'est la forme utilisée par les déploiements sérieux : elle est **infalsifiable**. Même si quelqu'un republie `socle/demo:1.0` avec un autre contenu, votre digest continue de désigner l'image que vous avez testée.

## Étape 8 — Transporter une image sans réseau

```
podman save -o /tmp/demo.tar demo-couches:1.0
ls -lh /tmp/demo.tar
tar -tf /tmp/demo.tar | head -n 4
```

**Observez** une archive de `8.4M` contenant des `<sha256>.tar` (les couches), un `.json` (la configuration) et un `manifest.json` : c'est le format historique `docker-archive`.

```
podman save --format oci-archive -o /tmp/demo-oci.tar demo-couches:1.0
tar -tf /tmp/demo-oci.tar | head -n 4
```

**Observez** cette fois `blobs/sha256/...` et `index.json` : la disposition **OCI**, celle que tous les outils (Docker, Podman, skopeo, Kubernetes) lisent.

Comparez avec `export`, qui travaille sur un **conteneur** :

```
podman run -d --name tmpx nginx:alpine
podman export tmpx | podman import - nginx-aplati:v1
podman image inspect --format '{{json .Config.Cmd}}' nginx-aplati:v1
podman run --rm nginx-aplati:v1
```

**Observez** `null`, puis l'erreur `crun: cannot find `` in $PATH` : l'image importée **ne sait plus quoi lancer**.

*Explication.* Retenez la règle : `save` / `load` pour une image, `export` / `import` jamais pour du déploiement.

```
podman rmi demo-couches:1.0 localhost:5000/socle/demo:1.1
podman load -i /tmp/demo.tar
```

**Observez** `Loaded image: localhost/demo-couches:1.0` : le tag est revenu avec l'archive.

---

## Nettoyage

---

```
podman rm -f -t 0 tmpx registry-labo
podman rmi nginx-aplati:v1 demo-couches:1.0 registry:2
podman images --format '{{.Repository}}:{{.Tag}}' | grep -E 'demo|aplati|registry'
rm -f /tmp/demo.tar /tmp/demo-oci.tar
```

Il peut rester l'image tirée par digest à l'étape 7 :

```
podman images --format 'table {{.ID}}\t{{.Repository}}' | grep localhost:5000
podman rmi <ID>
```

**Observez** qu'il ne reste que `docker.io/library/alpine` et `docker.io/library/nginx:alpine`, conservées pour la suite.

---

## Ce que vous devez pouvoir affirmer maintenant

---

- Un tag est une référence déplaçable ; le digest identifie un contenu. Podman stocke et affiche les noms complets, et préfixe les vôtres de `localhost/`.
- `podman history` et `podman image tree` révèlent où passe le poids d'une image et ce qu'elle partage.
- `podman tag` ne duplique rien ; `podman rmi` retire d'abord un nom.
- Un `push` ne transfère que les couches absentes du registry.
- Un registry est un simple service HTTP, montable en une commande — mais Podman exige TLS, sauf `--tls-verify=false` explicite.
- `export` / `import` détruit la configuration de l'image ; `save` / `load` la préserve, en format `docker-archive` ou `oci-archive`.